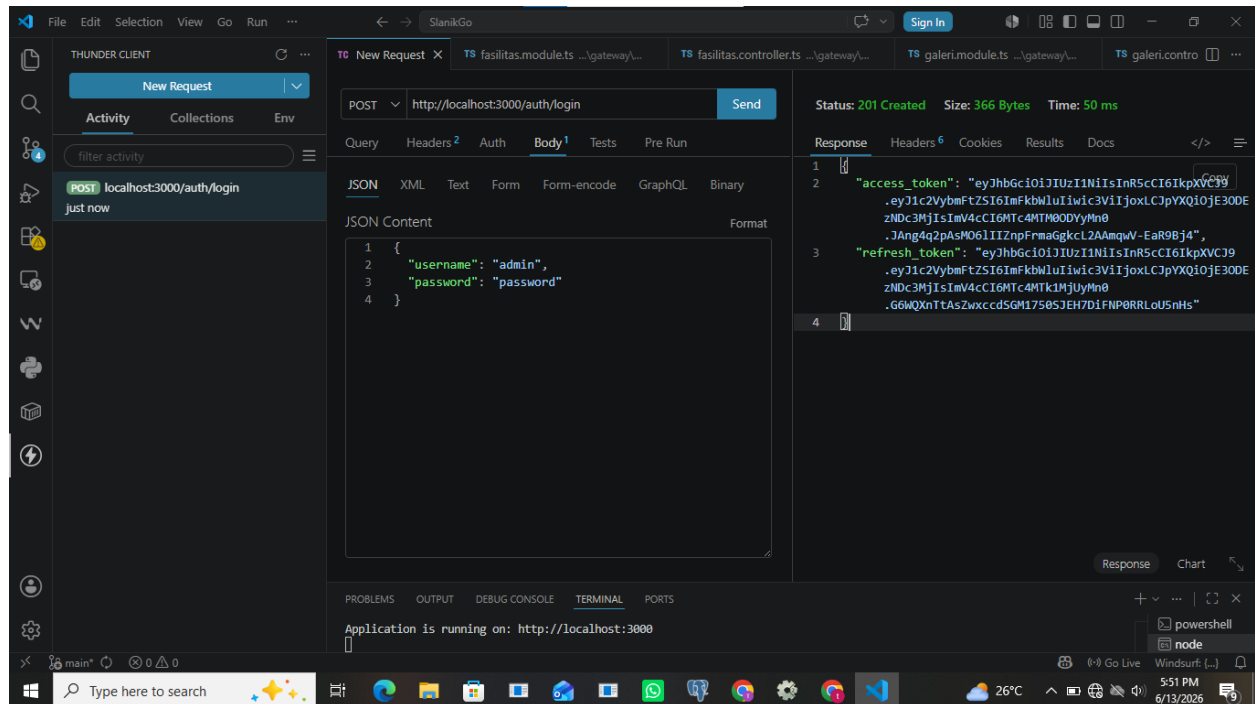


Nama: Trilia Gita Amanda

NPM: 23312053

Pengumpulan Backend tahap 3



1. Alamat API (Endpoint) dan Metode HTTP

- Metode: POST
- Digunakan karena proses ini mengirimkan data sensitif (username dan password) ke server untuk dibuatkan sesi barunya.
- URL: http://localhost:3000/auth/login
- Ini adalah alamat server lokal komputer kamu (localhost) yang berjalan pada port 3000. Jalur spesifiknya (path) adalah /auth/login yang menangani proses autentikasi.

2. Bagian Input (Body Request - JSON)

Di panel tengah bawah (tab Body -> JSON), kamu mengirimkan data berupa objek JSON berisi kredensial akun: {

```
"username": "admin",  
"password": "password"  
}
```

Data inilah yang dikirim ke server untuk diperiksa apakah akun tersebut terdaftar dan cocok.

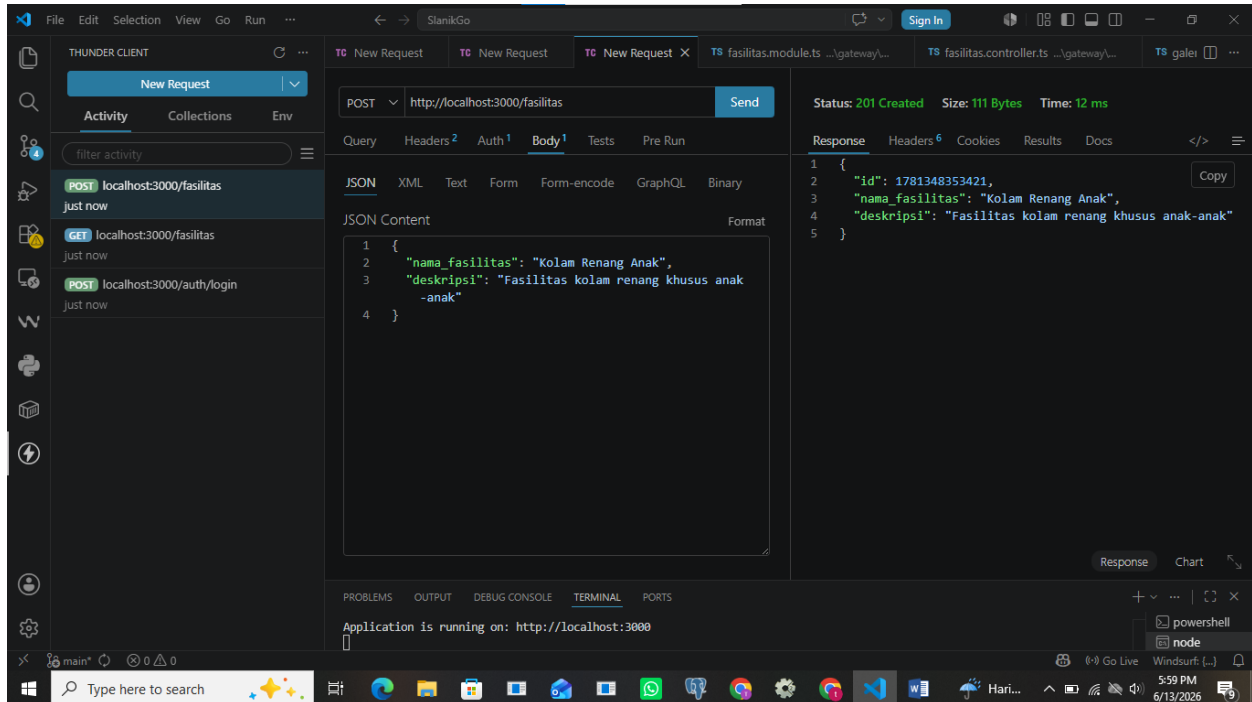
3. Hasil Respon dari Server (Response)

Di sebelah kanan, setelah menekan tombol Send, server memberikan balasan:

- Status: 201 Created * Artinya server berhasil memproses permintaan dan berhasil membuat data/token baru untuk sesi login ini.
- Ukuran & Waktu: Ukuran datanya 366 Bytes dan diproses sangat cepat dalam waktu 50 ms.

Isi Respon (Data JSON):

- access_token: Token utama (JWT) yang digunakan oleh aplikasi depan (frontend) untuk mengakses halaman-halaman yang terkunci/butuh izin akses.
- refresh_token: Token cadangan yang berfungsi untuk memperbarui access_token secara otomatis jika masa berlakunya sudah habis, tanpa memaksa pengguna untuk login ulang.



1. Endpoint dan Metode HTTP yang Digunakan

- Metode: POST
- Sesuai dengan standar REST API, metode POST di sini digunakan untuk mengirimkan data baru yang akan disimpan ke dalam database.
- URL: http://localhost:3000/fasilitas
- Jalur (path) yang dituju adalah /fasilitas. Ini berarti kamu sedang mengakses fitur atau controller yang bertanggung jawab mengelola data fasilitas aplikasi kamu.

2. Data yang Dikirim (Body Request - JSON)

Di panel tengah, kamu mengirimkan informasi objek berupa fasilitas yang ingin ditambahkan: JSON

```
{
  "nama_fasilitas": "Kolam Renang Anak",
  "deskripsi": "Fasilitas kolam renang khusus anak-anak"
}
```

3. Respon dari Server (Response Success)

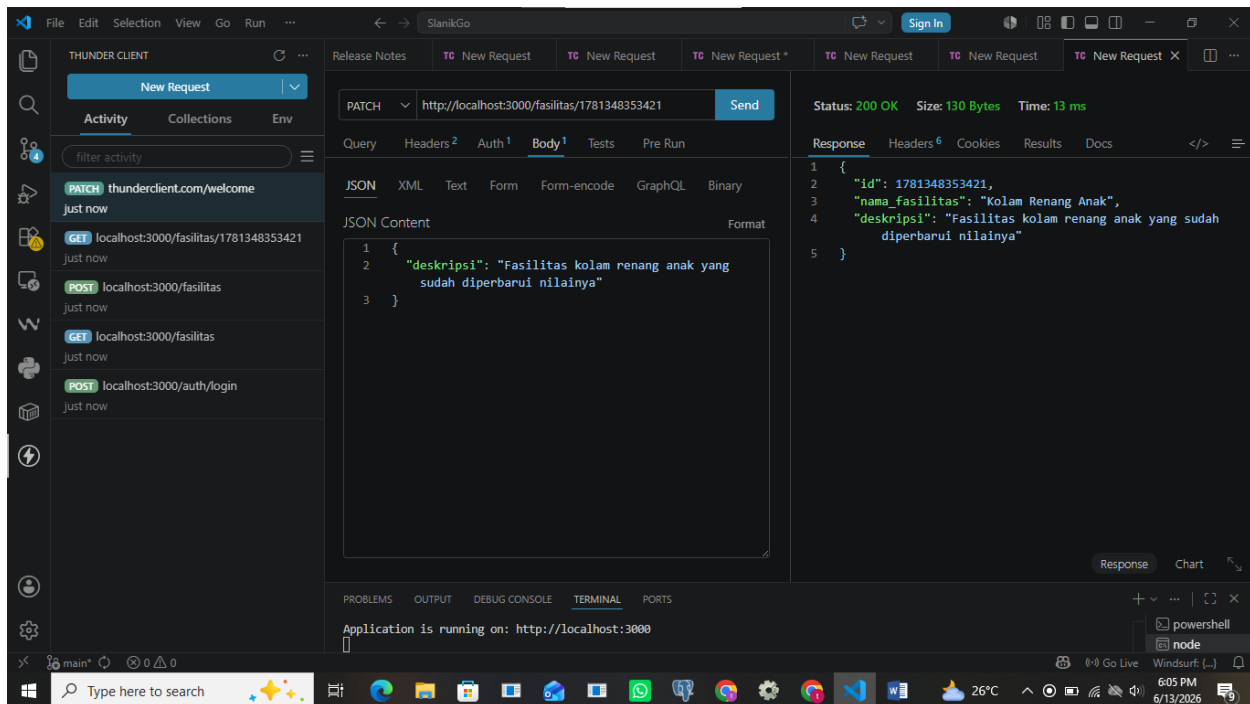
Di panel sebelah kanan, server backend memberikan respon balik yang menandakan operasi berhasil:

- Status: 201 Created
- Status HTTP ini menandakan bahwa data baru berhasil dibuat dan disimpan di sisi server/database.
- Waktu Proses: 12 ms
- Proses eksekusi ini berjalan sangat cepat (hanya 12 milidetik).

Isi Respon:

- JSON


```
{
  "id": 1781348353421,
  "nama_fasilitas": "Kolam Renang Anak",
  "deskripsi": "Fasilitas kolam renang khusus anak-anak"
}
```
- Server mengembalikan data yang baru saja kamu kirim, tetapi sekarang sudah dilengkapi dengan properti "id".
- "id": 1781348353421 bertindak sebagai Primary Key unik yang digenerate otomatis oleh sistem (menggunakan timestamp atau ID unik lainnya) untuk membedakan data kolam renang ini dengan fasilitas lainnya nanti.



1. Endpoint dan Metode HTTP yang Digunakan

- Metode: PATCH
- Berbeda dengan PUT yang biasanya mengganti seluruh data objek, metode PATCH digunakan dalam standar REST API untuk mengubah sebagian properti saja tanpa mengganggu data properti lainnya yang tidak dikirim.
- URL: http://localhost:3000/fasilitas/1781348353421

- Kamu menyertakan ID spesifik (1781348353421) di akhir URL (disebut sebagai Path Parameter). Ini memberi tahu server backend data fasilitas mana yang ingin kamu edit. ID ini cocok dengan data kolam renang anak yang kamu buat di langkah sebelumnya.
2. Data yang Dikirim (Body Request - JSON)
- Di panel tengah, kamu hanya mengirimkan satu buah properti yang ingin diganti:
 - JSON

```
{
  "deskripsi": "Fasilitas kolam renang anak yang sudah diperbarui nilainya"
}
```
 - Perhatikan bahwa kamu tidak mengirimkan properti "nama_fasilitas", karena kamu memang hanya berniat memperbarui teks deskripsinya saja.
3. Respon dari Server (Response Success)

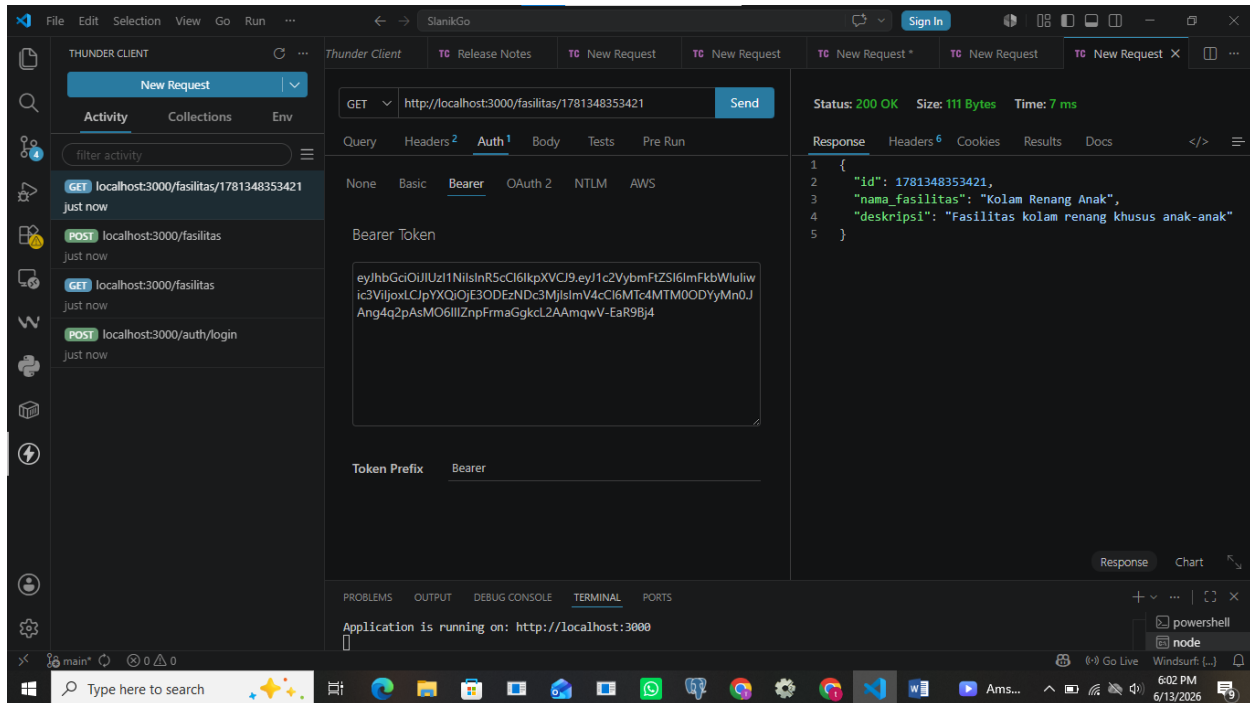
Di panel sebelah kanan, server memberikan respon balik yang menandakan pembaruan sukses:

- Status: 200 OK
- Status HTTP ini menandakan bahwa permintaan berhasil diproses dan data berhasil diperbarui di database.

Isi Respon:

- JSON

```
{
  "id": 1781348353421,
  "nama_fasilitas": "Kolam Renang Anak",
  "deskripsi": "Fasilitas kolam renang anak yang sudah diperbarui nilainya"
}
```
- Server mengembalikan objek data yang utuh setelah diperbarui.
- Bisa dilihat bahwa nilai "nama_fasilitas" tetap mempertahankan nilai lamanya yaitu "Kolam Renang Anak", sedangkan nilai "deskripsi" telah sukses berubah sesuai dengan input baru yang kamu kirimkan.



1. Endpoint dan Metode HTTP yang Digunakan

- Metode: GET
- Digunakan untuk membaca atau mengambil data dari server tanpa mengubah apa pun.
- URL: `http://localhost:3000/fasilitas/1781348353421`
- Kamu meminta data fasilitas yang memiliki ID spesifik "1781348353421" (Kolam Renang Anak).

2. Bagian Autentikasi (Tab Auth - Bearer Token)

- Ini adalah bagian paling penting pada gambar ini. Berbeda dengan pengujian sebelumnya di mana membuka tab Body, ini membuka tab Auth:
- Tipe: Bearer
- Memilih metode Bearer Token, yang merupakan standar industri untuk mengamankan REST API menggunakan JSON Web Token (JWT).
- Bearer Token Box:
- Memasukkan string panjang acak (eyJhbGciOi...) ke dalam kotak tersebut. Token ini adalah access_token yang kamu dapatkan sebagai respon sukses saat melakukan POST login di gambar pertama.
- Fungsinya: Sebagai "surat izin masuk". Server backend kamu mendeteksi token ini di bagian Header permintaan, memvalidasinya, dan karena tokennya sah, server mengizinkan melihat data tersebut. Jika token ini kosong atau salah, server biasanya akan menolak dengan status 401 Unauthorized.

3. Respon dari Server (Response Success)

Di panel sebelah kanan, server memberikan respon sukses:

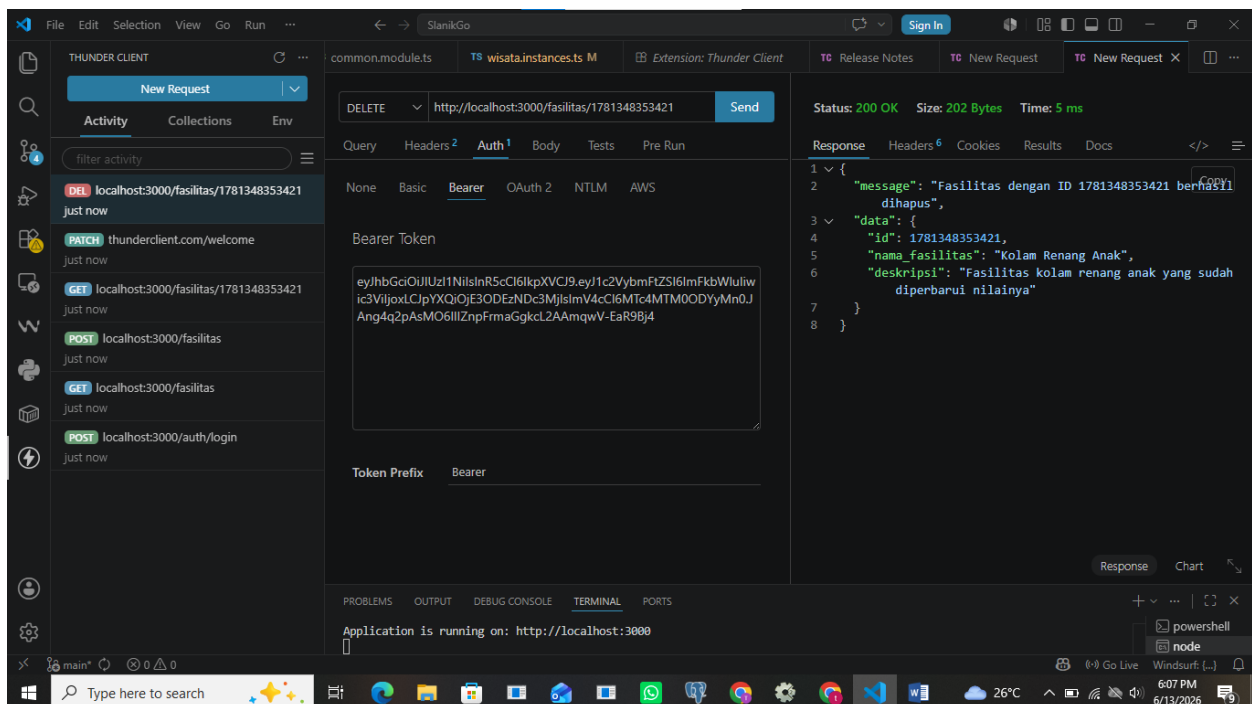
- Status: 200 OK
- Status HTTP ini menandakan permintaan pembacaan data berhasil diproses.

- Waktu Proses: 7 ms
- Proses pengambilan data dari database sangat cepat, hanya memakan waktu 7 milidetik.

Isi Respon:

- JSON


```
{
  "id": 1781348353421,
  "nama_fasilitas": "Kolam Renang Anak",
  "deskripsi": "Fasilitas kolam renang khusus anak-anak"
}
```
- Server berhasil mengembalikan data fasilitas yang kamu minta secara utuh dan spesifik berdasarkan ID-nya. (Catatan: teks deskripsinya kembali ke nilai awal, kemungkinan gambar ini diambil sesaat sebelum kamu melakukan uji coba PATCH/Update di gambar ketiga, atau datanya sempat di-reset).



1. Endpoint dan Metode HTTP yang Digunakan

- Metode: DELETE
- Sesuai dengan standar arsitektur REST API, metode DELETE digunakan khusus untuk menghancurkan atau menghapus data yang ada di server/database.
- URL: `http://localhost:3000/fasilitas/1781348353421`
- Mengirimkan perintah hapus ke jalur `/fasilitas/` dengan menyertakan ID data target (1781348353421) di ujung URL. Ini memastikan server hanya menghapus satu data spesifik (Kolam Renang Anak) dan tidak menghapus data lainnya.

2. Autentikasi Keamanan (Tab Auth)

- Sama seperti pada proses pembaruan dan pengambilan data sebelumnya, fitur hapus data ini bersifat sensitif. Oleh karena itu, menyertakan Bearer Token yang valid di tab Auth.
- Tanpa token ini, server backend akan menolak tindakan penghapusan demi keamanan data aplikasi.

3. Respon Sukses dari Server (Response)

Di panel sebelah kanan, server backend memberikan respon konfirmasi bahwa data telah sukses dihapus:

- Status: 200 OK
- Menandakan bahwa permintaan penghapusan diproses dengan sukses oleh sistem.
- Waktu Proses: 5 ms
- Proses eksekusi penghapusan di database berjalan sangat instan (hanya 5 milidetik).

Isi Respon (Data JSON):

- JSON

```
{
  "message": "Fasilitas dengan ID 1781348353421 berhasil dihapus",
  "data": {
    "id": 1781348353421,
    "nama_fasilitas": "Kolam Renang Anak",
    "deskripsi": "Fasilitas kolam renang anak yang sudah diperbarui nilainya"
  }
}
```
- Server memberikan umpan balik berupa "message" konfirmasi yang rapi.
- Server juga mengembalikan struktur data terakhir ("data") yang baru saja dihapus sebagai bukti objek mana yang telah dihilangkan dari database.